Integrated Case Study:
Deep Learning for Cyber Security

Phishing URL Detection Using Character-Level Embeddings and LSTM

Student: Akansha

Course: BTech CSE Core A

Notebook printout with implementation, architecture, training, evaluation, and deployment discussion.

```
Integrated Case Study: Deep Learning for Phishing URL Detection
Case study focus: character-level LSTM/BiLSTM URL classification, motivated by real-world anti-ph
```

services such as Google Safe Browsing. This notebook is a reproducible educational prototype, not Google's

internal implementation.

Dataset

```
Place a CSV at ./data/urls.csv with columns url and label (benign/legitimate = 0,
phishing/malicious = 1). If unavailable, a small synthetic URL dataset is generated to test the p
```

results must not be reported as real-world performance.

```
# If required:
# %pip install numpy pandas matplotlib scikit-learn tensorflow
```

```
import os, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
```

```
SEED=42
np.random.seed(SEED); random.seed(SEED); tf.random.set_seed(SEED)
print("TensorFlow:",tf.__version__)
```

1. Load labelled URL data

Use a trusted research dataset and review its license and collection date. Do not visit URLs from the dataset.

This task only processes URL strings as text.

```
DATA_PATH="./data/urls.csv"
USE_SYNTHETIC_IF_MISSING=True
```

```
def synthetic_urls(n=3000):
    rng=np.random.default_rng(SEED)
    benign_domains=["example.com","wikipedia.org","python.org","university.edu","github.com","d
    ocs.example.org"]
    phish_domains=["secure-login-example.test","account-verify-example.test","signin-alert-
    example.test","update-payment-example.test"]
    rows=[]
    for i in range(n):
        is_bad=int(rng.random()<0.35)
        if is_bad:
            dom=rng.choice(phish_domains)

    path=rng.choice(["/login/verify","/account/update","/security/check","/signin/session"])
            url="http://"+dom+path+"?id="+str(rng.integers(1000,999999))
        else:
            dom=rng.choice(benign_domains)
            path=rng.choice(["/","/about","/docs","/learn","/products"])
            url="https://"+dom+path
        rows.append((url,is_bad))
    return pd.DataFrame(rows,columns=["url","label"])

if os.path.exists(DATA_PATH):
    df=pd.read_csv(DATA_PATH)
```

```
    df.columns=[str(c).strip().lower() for c in df.columns]
    if not {"url","label"}.issubset(df.columns):
        raise ValueError("CSV must contain url and label columns")
else:
    if not USE_SYNTHETIC_IF_MISSING: raise FileNotFoundError(DATA_PATH)
    print("WARNING: using synthetic demonstration URLs, not a real benchmark dataset.")
    df=synthetic_urls()
df=df.dropna(subset=["url","label"]).copy()
df["url"]=df["url"].astype(str).str.strip()
def encode_label(v):
    s=str(v).strip().lower()
    if s in {"1","phishing","phish","malicious","bad","true"}: return 1
    if s in {"0","benign","legitimate","legit","good","false"}: return 0
    try: return int(float(s)>0)
    except: return 1 if ("phish" in s or "mal" in s) else 0
df["label"]=df["label"].map(encode_label).astype(int)
df=df[df["url"].str.len()>0].drop_duplicates(subset=["url"])
print(df.shape,df["label"].value_counts().to_dict())

df.head()
```

2. Character-level tokenisation and padding

Each URL is converted into a sequence of character IDs. The vocabulary contains observed characters plus an

unknown token. Sequences are truncated or padded to a fixed maximum length. Splitting occurs before

```
vocabulary fitting, so the held-out test set does not determine the character vocabulary.
urls=df["url"].tolist()
labels=df["label"].to_numpy(dtype=np.int32)
# Stratified split for URL classification; use domain-disjoint splits for stronger evaluation.
u_train,u_temp,y_train,y_temp=train_test_split(urls,labels,test_size=.30,random_state=SEED,stra
    tify=labels)
u_val,u_test,y_val,y_test=train_test_split(u_temp,y_temp,test_size=.50,random_state=SEED,strati
    fy=y_temp)

MAX_LEN=200
vocab=sorted(set("".join(u_train)))
char_to_id={ch:i+2 for i,ch in enumerate(vocab)} # 0=padding, 1=unknown
VOCAB_SIZE=len(char_to_id)+2
def encode_url(url):
    ids=[char_to_id.get(ch,1) for ch in str(url)[:MAX_LEN]]
    return ids+[0]*(MAX_LEN-len(ids))
X_train=np.asarray([encode_url(u) for u in u_train],dtype=np.int32)
X_val=np.asarray([encode_url(u) for u in u_val],dtype=np.int32)
X_test=np.asarray([encode_url(u) for u in u_test],dtype=np.int32)
print("Vocabulary size:",VOCAB_SIZE,"sequence length:",MAX_LEN)
print("Train/validation/test:",X_train.shape,X_val.shape,X_test.shape)
```

3. Model architecture

The embedding layer maps character IDs to vectors. A Bidirectional LSTM captures local and longer-range

character patterns across the URL. Global max pooling summarises the sequence, followed by dropout and a

sigmoid output for binary classification. This is a compact prototype; a production system would need robust

data collection, calibration, monitoring, and safe handling of false positives.

```
from IPython.display import Image, display
display(Image(filename="phishing_lstm_architecture.png"))
```

```
def build_model():
    inp=layers.Input(shape=(MAX_LEN,),dtype="int32")
    x=layers.Embedding(VOCAB_SIZE,32,mask_zero=True)(inp)
    x=layers.Bidirectional(layers.LSTM(48,return_sequences=True))(x)
    x=layers.GlobalMaxPooling1D()(x)
    x=layers.Dropout(.35)(x)
    x=layers.Dense(32,activation="relu")(x)
    out=layers.Dense(1,activation="sigmoid")(x)
    model=Model(inp,out,name="Character_BiLSTM_Phishing_Detector")
    model.compile(optimizer="adam",loss="binary_crossentropy",metrics=["accuracy"])
    return model
model=build_model()
model.summary()
```

```
early=EarlyStopping(monitor="val_loss",patience=3,restore_best_weights=True,verbose=1)
history=model.fit(
    X_train,y_train,validation_data=(X_val,y_val),
    epochs=15,batch_size=64,callbacks=[early],verbose=1
)
fig,ax=plt.subplots(1,2,figsize=(12,4))
ax[0].plot(history.history["loss"],label="train")
ax[0].plot(history.history["val_loss"],label="validation")
ax[0].set_title("Loss"); ax[0].set_xlabel("Epoch"); ax[0].legend()
ax[1].plot(history.history["accuracy"],label="train")
ax[1].plot(history.history["val_accuracy"],label="validation")
```

```
ax[1].set_title("Accuracy"); ax[1].set_xlabel("Epoch"); ax[1].legend()
plt.tight_layout(); plt.show()
```

```
prob=model.predict(X_test,verbose=0).ravel()
pred=(prob>=0.5).astype(int)
metrics={
 "Accuracy":accuracy_score(y_test,pred),
 "Precision":precision_score(y_test,pred,zero_division=0),
 "Recall":recall_score(y_test,pred,zero_division=0),
 "F1-score":f1_score(y_test,pred,zero_division=0)
}
print(pd.Series(metrics))
print("\\nConfusion matrix:\\n",confusion_matrix(y_test,pred))
print("\\nClassification report:\\n",classification_report(y_test,pred,target_names=["Benign","
    Phishing"],zero_division=0))
```

4. Deployment-oriented interpretation

A URL classifier can score URLs at browser, email gateway, DNS proxy, or security-service ingestion points.

A production service should combine model scores with reputation feeds, URL/domain age, redirects, page

content, and user reporting where legally and technically appropriate. High-risk decisions should be calibrated

against a validation set and monitored for drift.

```
Important evaluation caveat: a random stratified split can leak domain patterns across partitions
```

generalisation. For a stronger result, split by registered domain or time, and report metrics on a later time

```
period. Accuracy alone is not sufficient: phishing detection should consider recall (missed phish
precision (false alarms on legitimate sites).
```

References

- Google Safe Browsing, official overview: https://safebrowsing.google.com/
- Saxe, J. & Berlin, K. (2017). *eXpose: A Character-Level Convolutional Neural Network with Embe
For Detecting Malicious URLs, File Paths and Registry Keys*. arXiv:1702.08568.
- URLNet: Le, H. et al. (2018). *URLNet: Learning a URL Representation with Deep Learning for Mal
URL Detection*. arXiv:1802.03162.
- Hochreiter, S. & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8),

1735–1780.